

Design and Evaluation of a Distributed Hybrid System for Low-Latency Semantic and Structured Information Retrieval

Project Proposal [Draft]

Devang Chauhan

Msc Computer Science, University of London

Table of Contents

Introduction.....	3
Background Research.....	3
Problem	4
Aims and Objectives.....	4
Aims.....	4
Objectives.....	4
Development Plan	5
API.....	5
Information Stores	5
Extension.....	6
Embedding	6
References	7

Introduction

Since the introduction of large language models (LLM), there has been a sudden increase of retrieval systems, where information is retrieved rapidly from knowledge stores to ground the LLM responses in the most relevant information. It is important that such information is retrieved fast and it is relevant to a user query. Also, due to the size of the information being generated by users, in many scenarios, such retrieval system have to be distributed.

The aim of this project is to evaluate how different orchestration and routing strategies affect latency, scalability, and retrieval quality in distributed hybrid retrieval systems. In this project, hybrid retrieval refers to combining semantic vector retrieval with structured relational querying within a unified orchestration.

This report begins by reviewing the work in this area. It then moves on to a clear presentation of the problem, followed by the aim and objectives, and a high level development plan.

Background Research

Xu et al. [1] created a distributed RAG (DRAG) system using a peer to peer (P2P) network of knowledge stores, and local LLMs. Their work, set in the context of healthcare proposes a novel algorithm that looks for information locally or in a peer. While it is a distributed system, they solely rely on vector embeddings as grounding information for LLMs. Such a system does not represent an organisational level retrieval system where vector embeddings are not sufficient and a hybrid approach is required.

While Guerraoui et al. [2] acknowledge the distributed nature of the knowledge stores in the model applications, they propose RAGROUTE, a neural network classifier to quickly help decide which knowledge store to hit based on the query. This approach is probabilistic, and requires retraining the model in case the number of data stores changes.

Yu et al. [3] recognise the need to go beyond embeddings in real world application, and consider relational data as well. They generate vector embeddings after chunking table schema and add these embeddings along with the other embeddings. So if semantic search returns table schema chunks they query relevant tables. It is unclear how they address the change in table schema and large tables that may not

fit on one virtual machine.

Shalaan et al. [4] introduce guided SQL (GSQL), where they combine rule based approaches with natural language processing (NLP), and vector embeddings to come up with a near deterministic natural language (NL) to SQL translation.

Problem

Retrieval augmented generation (RAG) has emerged as a pivotal approach to mitigate hallucinations in LLMs. While the RAG approach has captured the interest of the research community, most approaches either focus narrowly on embeddings or propose retrieval pipelines that rely on probabilistic routing or classification approaches, which can complicate explainability and reproducibility during system evaluation.

Also, at any large organization, any modern day RAG application cannot solely rely on embeddings. In order to address a wide range of queries, other data sources such as relational and graph stores are also required to be queried. Due to rapid nature of data generation at such organisations, such data stores are highly likely to be distributed across discreet nodes.

Aims and Objectives

Aims

Evaluating latency of a hybrid knowledge retrieval system under distributed conditions.

Objectives

1. Design a query orchestration framework as a REST API that will receive queries and will return responses.
2. Measure and evaluate the latency of relevant response across metric
 - Mean / P95 / P99 latency,
 - Recall
 - Precision
 - Normalised discounted cumulative gain
 - Known-item retrieval accuracy
 - Horizontal scalability
 - Replication trade-offs

Development Plan

API

The query orchestration framework will be a REST API written in Rust. This will potentially include;

- Rule-based query parsing and routing logic
- Evaluation of multiple retrieval strategies, including:
 - Broadcast querying
 - Selective shard querying
 - Filter-then-search
 - Search-then-filter
- Retrieval coordination
- Distributed shard coordination
- Replication and node-failure experimentation
- Bench marking and evaluation infrastructure
- A gold-standard query to result mapping for bench marking

Since it is a statically typed, compiled, and without a garbage collector language, Rust is chosen to develop this latency sensitive layer.

This will not be a full CRUD application. Since the focus is on the query orchestration, only a limited number of endpoints will be developed. E.g. query response, node health, etc.

Information Stores

The evaluation in this project is limited to a vector store and a relational data store.

Publicly available movie datasets will be used as the information storage layer. IMDB non-commercial datasets will be used for the relational tables, and the MovieLens dataset will be used for vector embeddings.

Persistent copies of the vector database and the relational database will be setup locally using Docker.

A stand-alone Python script will be developed locally to chunk, generate embedding, and push the embedding to the vector database.

Embedding

In this project there are two use-case for generating embeddings out of natural language are; generating vector embedding from the user query, and generating vector embedding for each of the chunks for building the vector database.

For both these cases, a pre-trained embedding model available via API will be used.

Extension

Local distributed deployment will form the primary evaluation environment, with cloud deployment explored as an extension.

References

- [1] C. Xu, Y. Miao, L. Gao, X. Zhen, Distributed Retrieval-Augmented Generation, arXiv, arXiv:2505.00443v1, 2025 [Online]. Available: <https://arxiv.org/html/2505.00443v1>. [Accessed: May. 05, 2026]
- [2] R. Guerraoui, R. Pires, A. Kermarrec, M. Randl, D. Petrescu, M. De vos, "Efficient Federated Search for Retrieval-Augmented Generation", in Proc. *EuroMLSys: 5th Workshop on Machine Learning Systems*, Apr. 01, 2025, pp. 74-81. [Online]. Available: <https://dl.acm.org/doi/10.1145/3721146.3721942>. [Accessed: May. 05, 2026]
- [3] X. Yu, P. Jian, and C. Chen, "TableRAG: A Retrieval-Augmented Generation Framework for Heterogeneous Document Reasoning" in Proc. *Conf. on Empirical Methods in Natural Language Processing*, Nov. 4-9, 2025, pp. 14063–14082. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.710.pdf>. [Accessed: May. 06, 2026]
- [4] H. S. Shalaan, T. H. A. Soliman and A. M. AbdelAziz, "G-SQL: A Schema-Aware and Rule-Guided Approach for Robust Natural Language to SQL Translation," in Proc. *IEEE Access*, vol. 13, 2025, pp. 158520-158534. [Online]. Available: <https://ieeexplore.ieee.org/document/11153939>. [Accessed: May. 08, 2026]